WebSocket Chat API Documentation

Connection

```
const ws = new WebSocket('ws://your-server/chat?token=your-refresh-token');
```

The WebSocket connection requires an active subscription. Authenticate using a valid refresh token as a URL parameter.

Message Format

All messages follow this basic structure:

```
{
    "type": "message_type",
    // Additional parameters based on message type
}
```

Available Endpoints

1. Send Chat Message

Type: `chat`
Description: Send a message to the AI assistant

Parameters:

- `content` (required) - string: Message content
- `conversationId` (optional) - UUID: Existing conversation ID. If not provided, creates new conversation

Example Request:

```
{
    "type": "chat",
    "conversationId": "123e4567-e89b-12d3-a456-426614174000",
    "content": "What is the capital of France?"
}
```

Response Stream:

```
{
    "type": "chat_stream",
    "conversationId": "123e4567-e89b-12d3-a456-426614174000",
    "chunk": "Paris is", // Streaming chunks of the response
    "done": false       // true when stream complete
}
```

2. Get Chat History

Type: `get_history`
Description: Retrieve message history for a conversation

Parameters:

- `conversationId` (required) - UUID: Conversation ID
- `limit` (optional) - number: Maximum number of messages (default: 20)
- `offset` (optional) - number: Number of messages to skip (for pagination)

Example Request:

```
{
    "type": "get_history",
    "conversationId": "123e4567-e89b-12d3-a456-426614174000",
    "limit": 20,
    "offset": 0
}
```

Response:

```
{
    "type": "history",
    "conversationId": "123e4567-e89b-12d3-a456-426614174000",
    "messages": [
        {
            "role": "user",
            "content": "What is the capital of France?",
            "created_at": "2024-02-08T10:30:00Z"
        },
        {
            "role": "assistant",
            "content": "Paris is the capital of France...",
            "created_at": "2024-02-08T10:30:01Z"
        }
    ]
}
```

3. Get All Conversations

Type: `get_conversations`
Description: List all conversations for the user

Example Request:

```
{
    "type": "get_conversations"
}
```

Response:

```
{
    "type": "conversations",
    "conversations": [
        {
            "id": "123e4567-e89b-12d3-a456-426614174000",
            "title": "Geography Discussion",
            "created_at": "2024-02-08T10:30:00Z",
```

```
            "updated_at": "2024-02-08T10:35:00Z",
            "last_message": "Paris is the capital of France...",
            "message_count": 2
        }
    ]
}
```

4. Rename Conversation

Type: `rename_conversation`
Description: Change the title of a conversation

Parameters:

- `conversationId` (required) - UUID: Conversation ID
- `title` (required) - string: New conversation title

Example Request:

```
{
    "type": "rename_conversation",
    "conversationId": "123e4567-e89b-12d3-a456-426614174000",
    "title": "French Geography"
}
```

Response:

```
{
    "type": "conversation_renamed",
    "conversation": {
        "id": "123e4567-e89b-12d3-a456-426614174000",
        "title": "French Geography",
        "updated_at": "2024-02-08T10:40:00Z"
    }
}
```

5. Delete Conversation

Type: `delete_conversation`
Description: Delete a conversation and all its messages

Parameters:

- `conversationId` (required) - UUID: Conversation ID

Example Request:

```
{
    "type": "delete_conversation",
    "conversationId": "123e4567-e89b-12d3-a456-426614174000"
}
```

Response:

```
{
    "type": "conversation_deleted",
```

```
    "conversationId": "123e4567-e89b-12d3-a456-426614174000",
    "success": true
}
```

Error Handling

When an error occurs, the server will respond with an error message:

```
{
    "type": "error",
    "error": "Error message description"
}
```

Common error scenarios:

- Invalid conversation ID
- Unauthorized access
- Server errors
- Rate limiting
- Invalid message format

WebSocket Events

The client should handle these WebSocket events:

```
ws.onopen = () => {
    console.log('Connected to chat server');
};

ws.onmessage = (event) => {
    const data = JSON.parse(event.data);
    // Handle different message types
    switch(data.type) {
        case 'chat_stream':
            // Handle streaming response
            break;
        case 'history':
            // Handle chat history
            break;
        case 'conversations':
            // Handle conversations list
            break;
        case 'error':
            // Handle error
            break;
    }
};

ws.onerror = (error) => {
    console.error('WebSocket error:', error);
};

ws.onclose = (event) => {
```

```
    console.log('Disconnected from chat server:', event.code, event.reason);
};
```